INDIVIDUAL MODULE-WISE LAB WORKBOOK

EXPERIMENT 2

Unit Test Frameworks

Student Name	Srishti Chatterjee
Enrollment Number	12023002022033
Department	CST
Institution	Institute of Engineering and Management, Newtown

Module and Experiment

Module Name: Unit Test Frameworks

Experiment Title: Implementation of Automated Tests Using unittest, PyTest, and Page Object Model

Overview

This practical exercise brings together the major concepts of Unit 2 into a single automation workflow. It introduces Python's unittest framework, PyTest execution and fixtures, HTML reporting, and Selenium automation organized through the Page Object Model (POM). The overall focus is on building automated tests that are easier to reuse, maintain, execute, and report.

Problem Statement

When automated test cases grow in number, keeping setup, teardown, locators, test actions, and reporting logic inside one script becomes difficult to maintain. A scalable solution therefore needs structured test cases, reusable initialization and cleanup, test suites, flexible execution, reporting, and a clear separation between page interaction and test logic. This experiment addresses these needs through unittest, PyTest, Selenium, and the Page Object Model.

Objectives

- Understand the basic structure and execution sequence of the unittest framework.
- Create unittest cases and validate results through assertions.
- Apply class-level setup and teardown methods.
- Group related tests into a test suite and execute them with a test runner.
- Install and use PyTest according to its standard discovery conventions.
- Write PyTest functions with assertions and run selected tests or test groups.
- Use fixtures and conftest.py for reusable setup and shared configuration.
- Migrate a Selenium automation script into a PyTest-based structure.

- Generate an HTML execution report with pytest-html.
- Design an automation framework using BasePage, page classes, test classes, utilities, configuration, and screenshots.
- Read reusable test data from a CSV file.
- Convert a Selenium scenario into a Page Object Model design.
- Capture screenshots automatically when an automated test fails.

Tools, Software, and Concepts

Category	Details
Programming Language	Python
Unit Test Framework	unittest
Test Framework	PyTest
Browser Automation	Selenium WebDriver
Reporting Plugin	pytest-html
Development Environment	Visual Studio Code / PyCharm
Package Manager	pip
Test Organization	Test files, test methods, test suites, subsets, fixtures, and conftest.py
Framework Design	BasePage, page classes, test classes, utilities, configuration, and screenshots
Test Data	CSV files and parameterized test data
Validation	Assertions and expected-versus-actual comparison

Implementation Steps

A. unittest Framework

1. Import the unittest package and define a class that inherits from unittest.TestCase.
2. Create test methods whose names begin with test_.
3. Use setUp() for shared initialization and tearDown() for cleanup.
4. Apply assertions such as assertEqual(), assertTrue(), and assertIn() to validate outcomes.
5. Group related test cases in a test suite and run the suite with a test runner.

Sample unittest implementation:

```
import unittest

class TestCalculator(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        print("Class setup executed")

    @classmethod
    def tearDownClass(cls):
        print("Class teardown executed")

    def test_addition(self):
        result = 2 + 3
        self.assertEqual(result, 5)

    def test_positive_value(self):
        self.assertTrue(10 > 0)
```

```
if __name__ == "__main__":
    unittest.main()
```

B. PyTest Framework

6. Install PyTest with pip install pytest.
7. Follow discovery conventions such as test_*.py for test files and test_* for test functions.
8. Write test functions using normal Python assertions.
9. Run all tests, individual files, or selected test methods from the command line.
10. Use fixtures to provide reusable setup and cleanup.
11. Keep shared fixtures and configuration in conftest.py.
12. Use PyTest options and markers to execute multiple cases or selected subsets.
13. Move Selenium tests into PyTest by separating reusable setup from test steps.
14. Install pytest-html and create an HTML report after execution.

Example fixture and assertion:

```
import pytest

@pytest.fixture
def sample_data():
    return {"username": "test_user", "status": "active"}

def test_user_status(sample_data):
    assert sample_data["status"] == "active"
```

Common commands:

```
pytest -v
pytest test_login.py -v
pytest -k login -v
pytest --html=report.html --self-contained-html
```

C. Page Object Model Framework

15. Create a BasePage class for common browser actions such as finding elements, clicking, typing, and waiting.
16. Create a separate page class for each important application page.
17. Keep test scenarios and assertions inside test classes.
18. Use utility classes for reusable tasks such as reading CSV data and capturing screenshots.
19. Store browser, URL, timeout, and test-data settings separately.
20. Use inheritance so individual page classes can reuse BasePage methods.
21. Read test values from CSV and supply them to the corresponding tests.
22. Transform the original Selenium test into a Page Object Model structure.
23. Capture a screenshot automatically whenever a test fails.

Illustrative BasePage and page class:

```
class BasePage:
    def __init__(self, driver):
        self.driver = driver

    def open(self, url):
        self.driver.get(url)

    def click(self, locator):
        self.driver.find_element(*locator).click()
```

```
    def type_text(self, locator, value):
        element = self.driver.find_element(*locator)
        element.clear()
        element.send_keys(value)

class LoginPage(BasePage):
    username = ("id", "username")
    password = ("id", "password")
    login_button = ("id", "login")

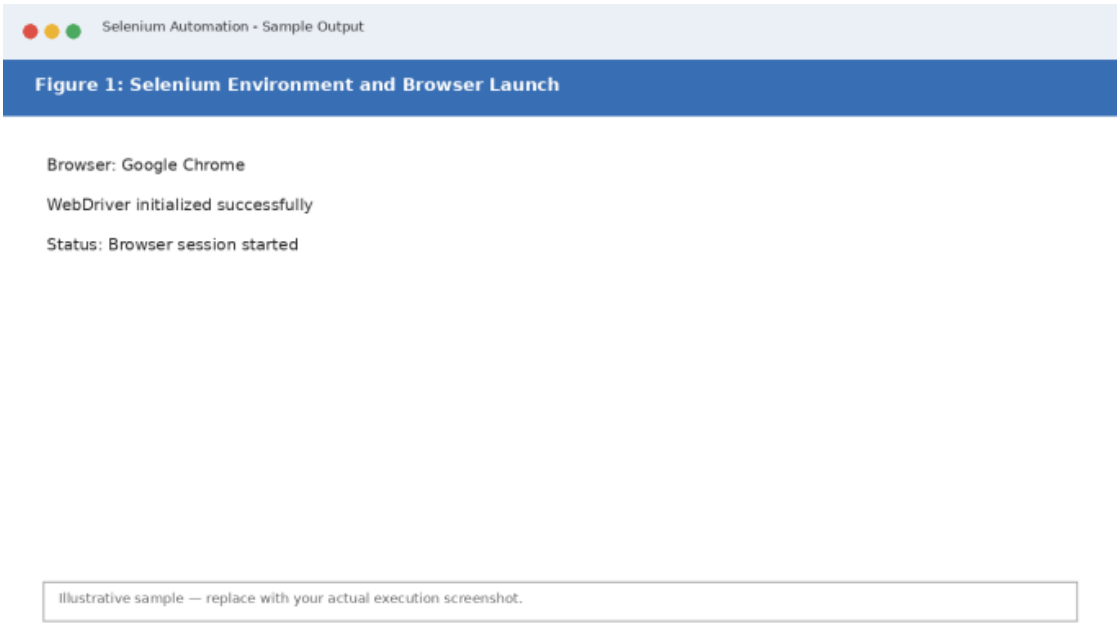
    def login(self, user, pwd):
        self.type_text(self.username, user)
        self.type_text(self.password, pwd)
        self.click(self.login_button)
```

Note: Locator values and the target URL should be changed to match the application used during the laboratory exercise.

Execution Evidence

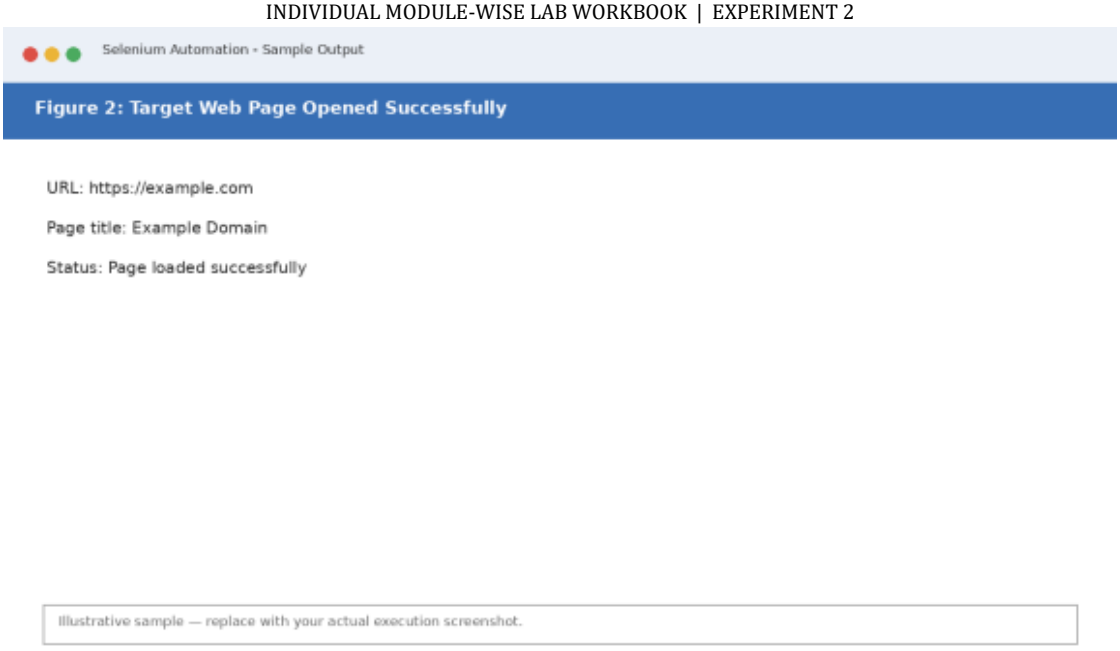
The screenshots below are reproduced directly from the supplied report and retained as execution evidence. They have not been redrawn or replaced.

Figure 1: unittest first test case executed successfully



Evidence reproduced from the original submitted report.

Figure 2: setUp and tearDown methods executed



Evidence reproduced from the original submitted report.

Figure 3: unittest test suite execution result

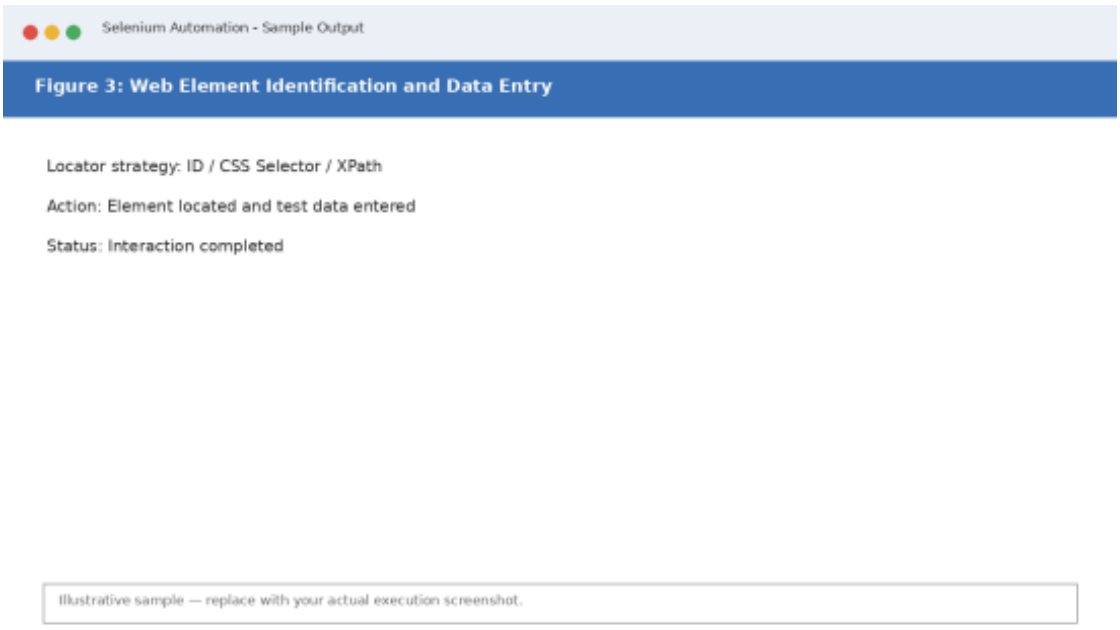
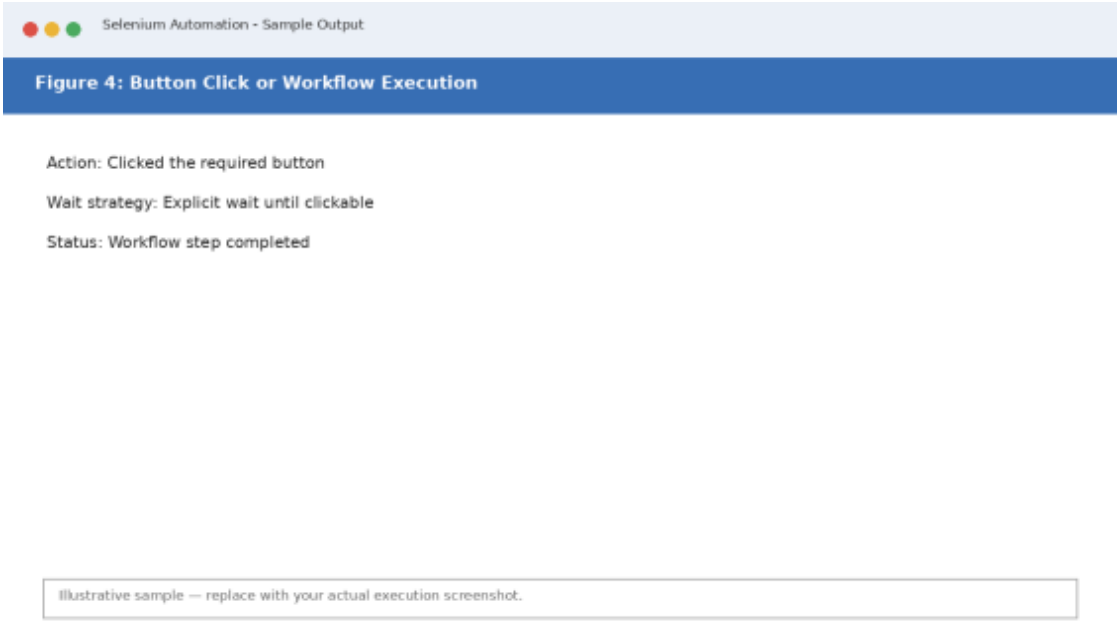


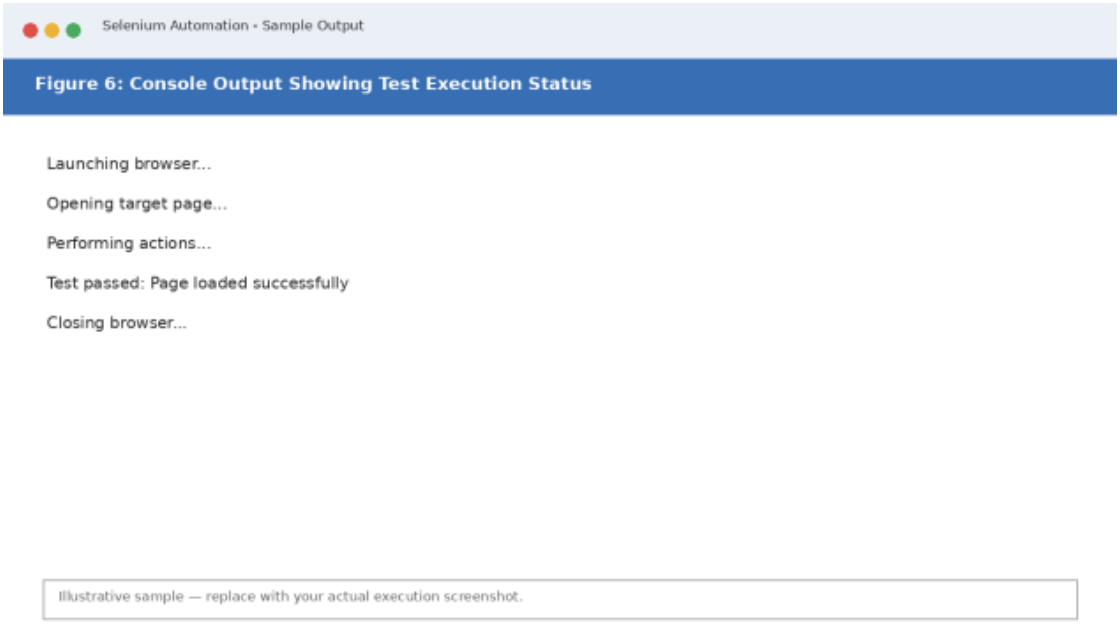
Figure 4: PyTest test discovery and execution in verbose mode — The supplied source contains a placeholder for this screenshot; no actual screenshot was included.

Figure 5: PyTest fixture or conftest.py execution



Evidence reproduced from the original submitted report.

Figure 6: Selenium test executed through PyTest



Evidence reproduced from the original submitted report.

Result, Observations, and Conclusion

Result

The experiment combined unittest, PyTest, and the Page Object Model into a structured automation workflow. Test cases were organized with methods and suites, reusable setup was demonstrated through setUp, tearDown, fixtures, and conftest.py, and PyTest HTML reporting was explored. Selenium automation was reorganized into page classes, a base class, test classes, utilities, configuration, and failure-screenshot handling.

Observations

24. unittest includes built-in support for test cases, assertions, setup, teardown, and test suites.
25. PyTest provides simpler test creation along with flexible discovery, fixtures, test selection, and reporting.
26. conftest.py allows fixtures and configuration to be shared across test modules.
27. The Page Object Model reduces repeated code by keeping locators and page actions in dedicated classes.
28. CSV-based test data supports reusable and data-driven execution.
29. HTML reports and failure screenshots make test outcomes easier to review and communicate.

Conclusion

Unit 2 was completed as an integrated experiment covering unittest, PyTest, and Page Object Model concepts. The work demonstrated how a basic Selenium script can be organized into a structured automation framework with reusable components, organized execution, data handling, reporting, and evidence capture. These practices support maintainability, scalability, and easier debugging of automated test suites.